CAPSTONE PROJECT REPORT

Web Application Penetration Testing and Incident Response Using DVWA

Application: Damn Vulnerable Web Application (DVWA)

Environment: Kali Linux Local / Isolated Laboratory

Target: 127.0.0.1

Prepared by

Shusmita Pal

B.Sc. (Hons) Advanced Networking & Cyber Security

Submission Date: 05 October 2026

All testing documented in this report was performed against the controlled DVWA laboratory environment.

1. Executive Summary

This capstone presents a controlled web-application security assessment and incident-response exercise using Damn Vulnerable Web Application (DVWA). Testing was performed only against the local target 127.0.0.1 in an isolated Kali Linux laboratory. The project combines reconnaissance, vulnerability testing, evidence collection, remediation and a simulated SQL Injection incident-response workflow.

The supplied evidence covers SQL Injection, reflected/stored XSS, CSRF, LFI/RFI, Burp Suite request analysis and security-header review. The capstone adds Nmap reconnaissance and a complete SQL Injection response cycle: detection from Apache access logs, containment, restoration of a prepared-statement implementation, recovery and verification.

Assessment Summary

Area	Result
Target	DVWA / 127.0.0.1
Recon	Nmap service/version + OS detection
Testing	SQLi, XSS, CSRF, LFI/RFI, Burp Suite
Incident	Simulated SQL Injection
Detection	Apache access log captured encoded SQLi request
Containment	SQLi module isolated
Remediation	Prepared-statement implementation restored
Recovery	Application path restored and secure code verified

2. Table of Contents

3. Introduction

4. Objectives

5. Scope & Rules of Engagement

6. Lab Environment & Tools

7. Network Architecture

8. Methodology

9. Reconnaissance & Nmap

10. Vulnerability Findings

11. SQL Injection

12. Cross-Site Scripting (XSS)

13. CSRF

14. LFI/RFI

15. Burp Suite & Security Headers

16. Incident Response

17. Findings & Risk Assessment

18. Mitigation & Recommendations

19. Conclusion

20. Evidence Appendix

3. Introduction

Web applications process untrusted input, maintain sessions and interact with databases. Weak input handling or missing request validation can create vulnerabilities including SQL Injection, XSS, CSRF and file-inclusion flaws. DVWA provides an intentionally vulnerable environment for demonstrating these concepts safely.

This capstone connects offensive testing with defensive operations: demonstrate a weakness, preserve evidence, detect related activity in logs, contain the affected component, apply remediation, restore service and verify the control.

4. Objectives

- Perform controlled reconnaissance against the local DVWA application.
- Demonstrate representative web-application security weaknesses using controlled lab evidence.
- Document impact and defensive controls for each finding.
- Detect a simulated SQL Injection attempt using Apache access logs.
- Contain the affected component and restore a secure prepared-statement implementation.
- Verify recovery and preserve evidence for the final report and presentation.

5. Scope & Rules of Engagement

In scope: DVWA running locally in Kali Linux; HTTP application testing; Nmap reconnaissance; controlled vulnerability demonstrations; Burp Suite request/response inspection; Apache log analysis; remediation verification.

Out of scope: Public websites, third-party systems, production infrastructure, and any system without authorization. All demonstrations described here were performed against the controlled local DVWA lab.

6. Lab Environment & Tools

Component	Use
Kali Linux	Security-testing workstation and local DVWA host
DVWA	Intentionally vulnerable web application
Apache	Web server; HTTP on port 80
MariaDB	Database service on port 3306
Nmap	Service/version and OS reconnaissance
Burp Suite	HTTP request/response inspection
Wireshark	Traffic observation capability
Firefox	Browser interaction with DVWA

7. Network Architecture

The following architecture represents the local isolated capstone environment. DVWA, Apache and MariaDB provide the application stack; Nmap and Burp Suite support controlled assessment; Apache access logs provide incident-detection evidence.

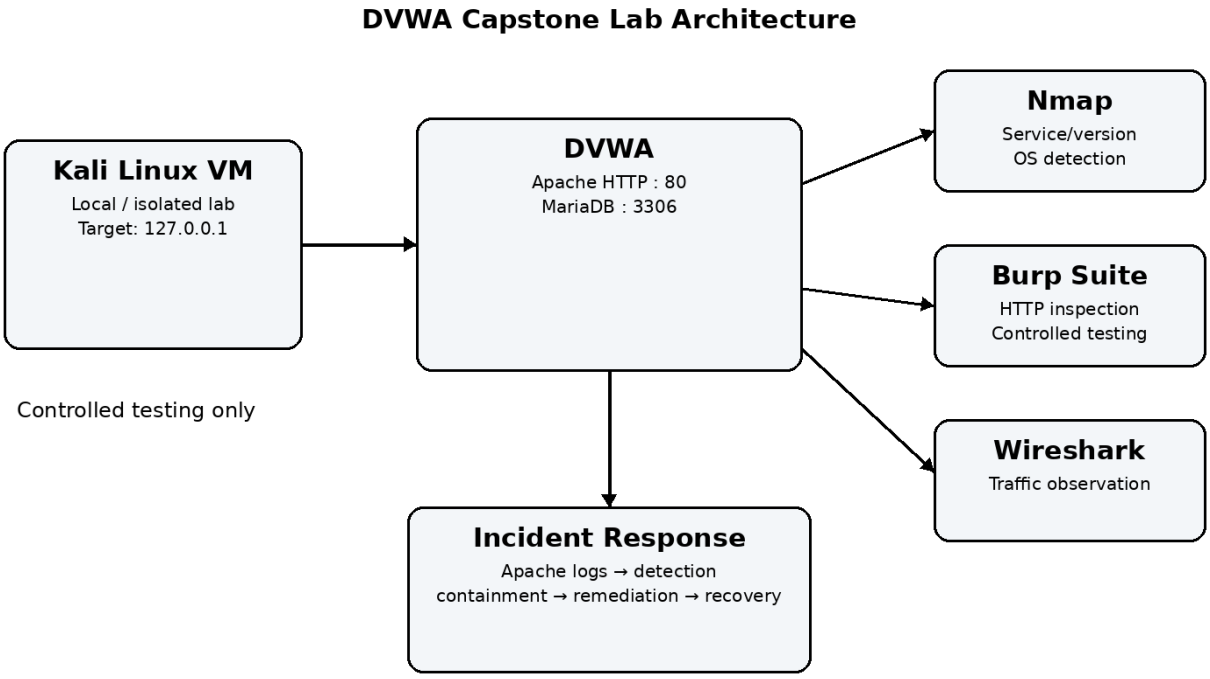


Figure 1. Capstone laboratory architecture and incident-response evidence flow.

8. Methodology

- **Reconnaissance:** Confirm the local application and enumerate exposed services with Nmap.
- **Vulnerability assessment:** Use controlled DVWA testing evidence for SQLi, XSS, CSRF, LFI/RFI and related web-security weaknesses.
- **HTTP analysis:** Use Burp Suite to inspect and manipulate controlled requests and responses.
- **Evidence collection:** Preserve screenshots, Nmap output and Apache log evidence.
- **Incident detection:** Search Apache logs for encoded SQLi indicators associated with the simulated test.
- **Containment:** Temporarily isolate the vulnerable SQLi module.
- **Eradication:** Restore the prepared-statement implementation.
- **Recovery:** Restore the application path and verify Apache and the secure code.

9. Reconnaissance & Nmap

Nmap service/version detection against 127.0.0.1 identified HTTP on port 80 as Apache httpd 2.4.62 and a database service on port 3306 identified as MariaDB 11.8.8 during fingerprinting. The OS-detection scan reported a Linux operating-system family and zero-hop network distance, consistent with the local target.

```
┌──(kali㉿kali)-[~]
└─$ nmap -sV 127.0.0.1
Starting Nmap 7.95 ( https://nmap.org ) at 2026-10-05 01:00 IST
Nmap scan report for localhost (127.0.0.1)
Host is up (0.0000060s latency).
Not shown: 998 closed tcp ports (reset)
PORT     STATE SERVICE VERSION
80/tcp   open  http    Apache httpd 2.4.62 ((Debian))
3306/tcp open  mysql?
1 service unrecognized despite returning data. If you know the service/version, please submit the following fingerprint at htt
ps://nmap.org/cgi-bin/submit.cgi?new-service :
SF-Port3306-TCP:V=7.95%I=7%D=10/5%Time=6AC2A947%P=x86_64-pc-linux-gnu%r(NU
SF:LL,64,"`\0\0\0\n11\.8\.8-MariaDB-1\x20from\x20Debian\0\r\0\0\x003vy\|\$
SF::6@\0\xfe\xff-\x02\0\xff\x81\x15\0\0\0\0\0=\0\0\0!!\$\.Z\\@iq}@f\0mys
SF:ql_native_password\0")%r(GenericLines,9C,"`\0\0\0\n11\.8\.8-MariaDB-1\x
SF:20from\x20Debian\0\r\0\0\x003vy\|\$:6@\0\xfe\xff-\x02\0\xff\x81\x15\0\0
SF:\0\0\0\0=\0\0\0!!\$\.Z\\@iq}@f\0mysql_native_password\x004\0\0\x01\xffj
SF:\x04#HY000Proxy\x20header\x20is\x20not\x20accepted\x20from\x20127\.0\.0
SF:\.1")%r(LDAPBindReq,64,"`\0\0\0\n11\.8\.8-MariaDB-1\x20from\x20Debian\0
SF:\x1e\0\0\0AAdSU\.'t\0\xfe\xff-\x02\0\xff\x81\x15\0\0\0\0\0=\0\0\x007T
SF:\"I\+9z~\)O3Q\0mysql_native_password\0")%r(afp,64,"`\0\0\0\n11\.8\.8-Ma
SF:riaDB-1\x20from\x20Debian\0\(\0\0\0LJnmVIJu\0\xfe\xff-\x02\0\xff\x81\x1
SF:5\0\0\0\0\0\0=\0\0\x0010=\"1pbyY33D\0mysql_native_password\0");

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
```

Figure 2. Nmap service/version detection against the local DVWA host.

```
┌──(kali㉿kali)-[~]
└─$ sudo nmap -sV -O 127.0.0.1
Starting Nmap 7.95 ( https://nmap.org ) at 2026-10-05 01:01 IST
Nmap scan report for localhost (127.0.0.1)
Host is up (0.000065s latency).
Not shown: 998 closed tcp ports (reset)
PORT     STATE SERVICE VERSION
80/tcp   open  http    Apache httpd 2.4.62 ((Debian))
3306/tcp open  mysql?
1 service unrecognized despite returning data. If you know the service/version, please submit the following fingerprint at htt
ps://nmap.org/cgi-bin/submit.cgi?new-service :
SF-Port3306-TCP:V=7.95%I=7%D=10/5%Time=6AC2A988%P=x86_64-pc-linux-gnu%r(NU
SF:LL,64,"`\0\0\0\n11\.8\.8-MariaDB-1\x20from\x20Debian\0\+\0\0\x007_Z%HzQ
SF:b\0\xfe\xff-\x02\0\xff\x81\x15\0\0\0\0\0=\0\0\0y\[<W!>VtdyUy\0mysql_n
SF:ative_password\0")%r(GenericLines,9C,"`\0\0\0\n11\.8\.8-MariaDB-1\x20fr
SF:om\x20Debian\0\+\0\0\x007_Z%HzQb\0\xfe\xff-\x02\0\xff\x81\x15\0\0\0\0\0
SF:\0=\0\0\0y\[<W!>VtdyUy\0mysql_native_password\x004\0\0\x01\xffj\x04#HY0
SF:00Proxy\x20header\x20is\x20not\x20accepted\x20from\x20127\.0\.0\.1")%r(
SF:LDAPBindReq,64,"`\0\0\0\n11\.8\.8-MariaDB-1\x20from\x20Debian\0<\0\0\0C
SF:vMZ_OMq\0\xfe\xff-\x02\0\xff\x81\x15\0\0\0\0\0=\0\0\0smHa/d-P,\+0Q\0m
SF:ysql_native_password\0")%r(afp,64,"`\0\0\0\n11\.8\.8-MariaDB-1\x20from\
SF:x20Debian\0F\0\0\0\.S7Y<\^G&\0\xfe\xff-\x02\0\xff\x81\x15\0\0\0\0\0=\
SF:0\0\0\)8\|,a%5x!5\(f\0mysql_native_password\0");
Device type: general purpose
Running: Linux 2.6.X|5.X
OS CPE: cpe:/o:linux:linux_kernel:2.6.32 cpe:/o:linux:linux_kernel:5 cpe:/o:linux:linux_kernel:6
OS details: Linux 2.6.32, Linux 5.0 - 6.2
Network Distance: 0 hops

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
```

Figure 3. Nmap service/version and OS detection.

10. Vulnerability Findings

The findings below consolidate the controlled evidence supplied for this capstone. Risk values are qualitative project-prioritization ratings, not vendor CVSS scores.

ID	Finding	Risk	Observed Evidence / Impact	Primary Mitigation
V-01	SQL Injection	High	Crafted input altered query behavior and exposed additional database information, validating	Prepared statements; input validation; least-privilege DB.
V-02	Reflected / Stored XSS	Medium	Script payloads executed in reflected and stored scenarios.	Context-aware output encoding; CSP.
V-03	CSRF	Medium	Password-change request captured/inspected in Burp, demonstrating state-changing risk.	Anti-CSRF tokens; SameSite; origin checks.
V-04	LFI/RFI	High	File-inclusion modules were exercised and evidence captured.	Strict allowlists; path canonicalization; disable unnecessa
V-05	Security-header weaknesses	Medium	Supplied header review showed missing/weak browser security controls.	Establish and continuously validate a security-header bas

11. SQL Injection

The SQL Injection module was tested with crafted input intended to change backend query logic. The supplied evidence shows vulnerable behavior and database records returned by the application. This demonstrates that untrusted input could influence SQL query execution in the vulnerable configuration.

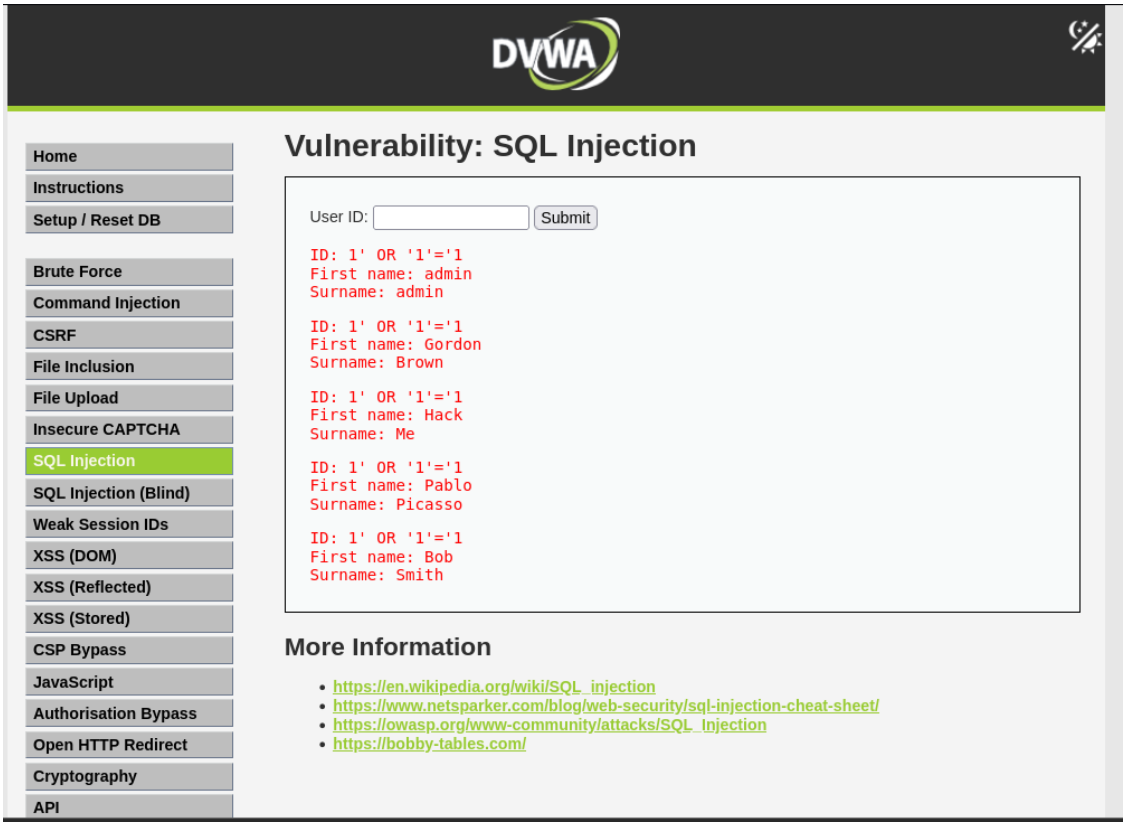


Figure 4. SQL Injection behavior in DVWA.

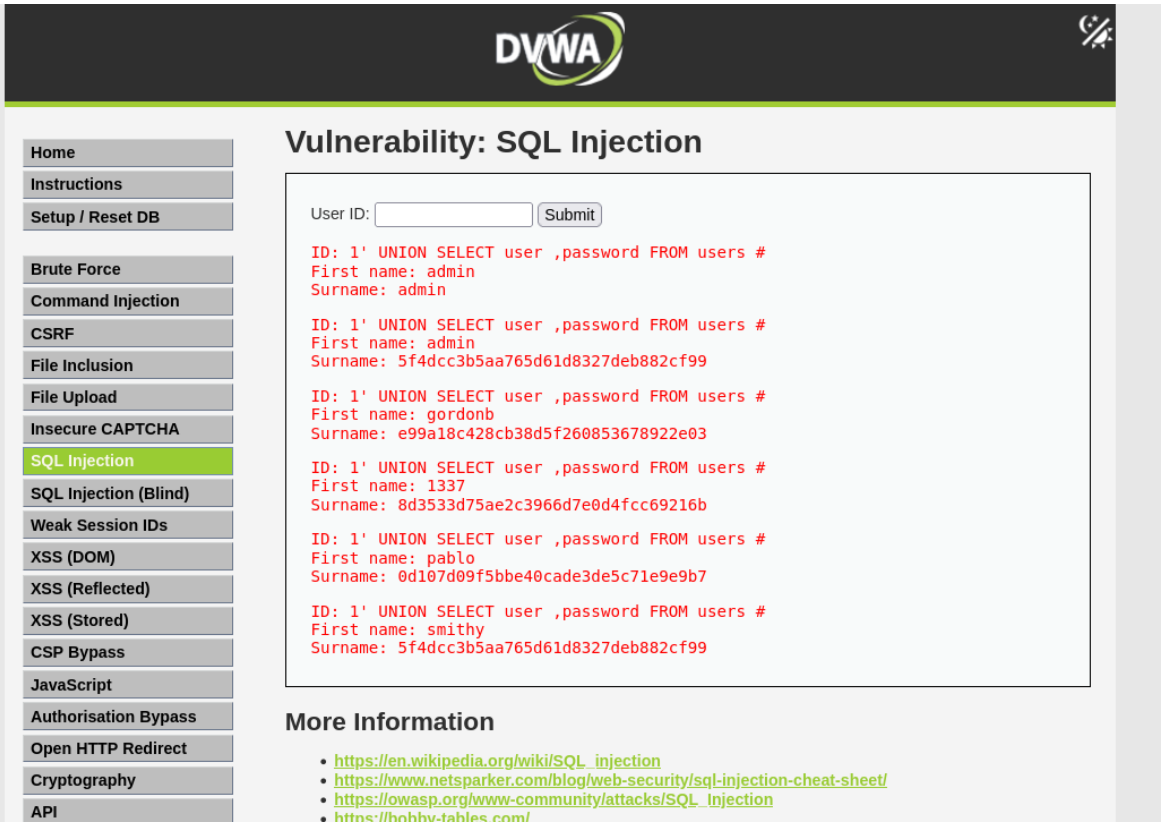


Figure 5. SQL Injection evidence exposing additional database information.

Impact: SQL Injection can permit unauthorized retrieval or manipulation of database information and may lead to broader compromise depending on privileges and architecture.

Mitigation: Use prepared statements/parameterized queries, server-side validation, least-privilege database accounts and safe error handling.

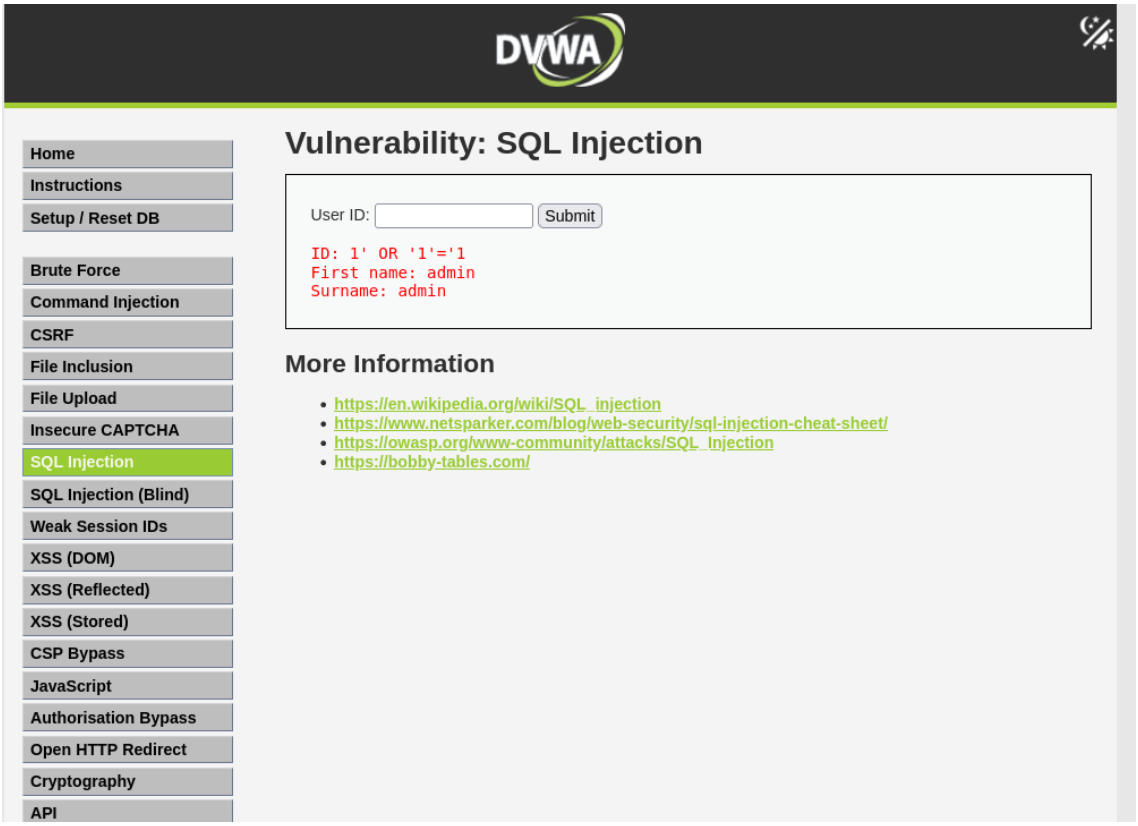


Figure 6. SQL Injection mitigation evidence.

12. Cross-Site Scripting (XSS)

The supplied evidence covers reflected and stored XSS behavior. The reflected case demonstrates script execution in the response context; the stored case demonstrates execution when stored content is rendered.

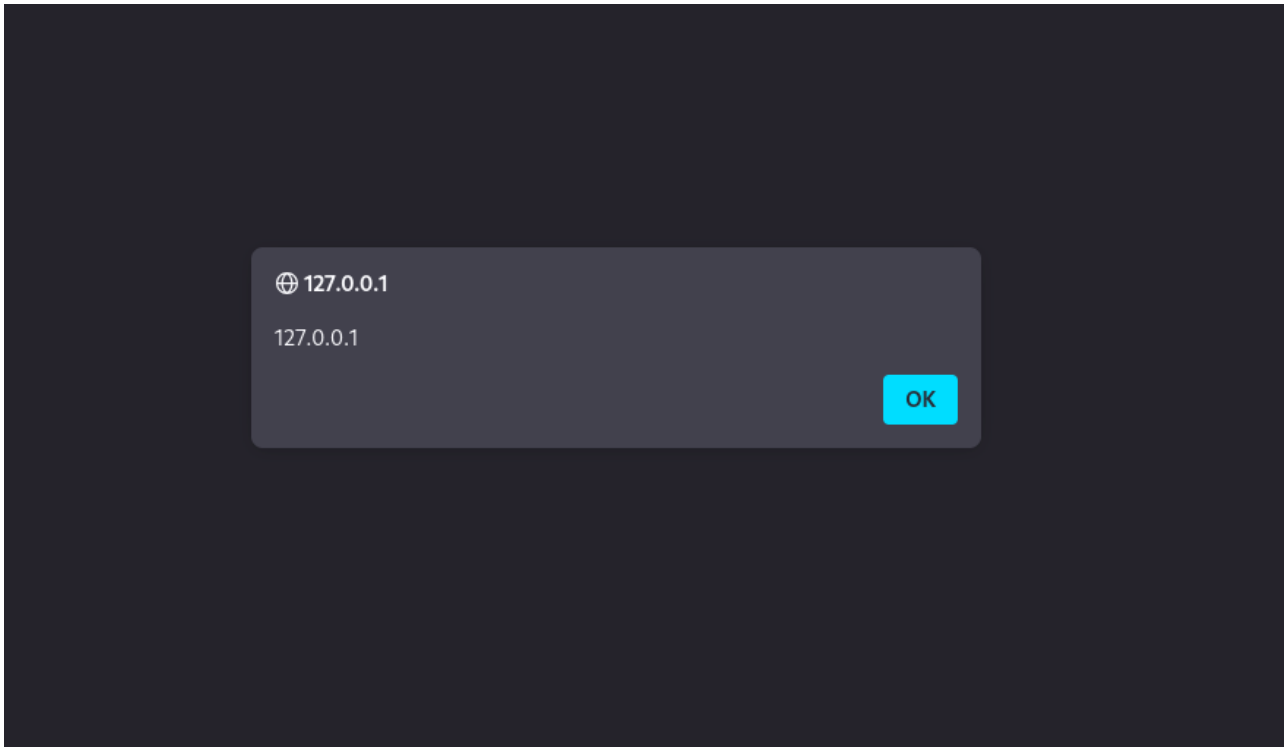


Figure 7. Reflected XSS execution evidence.

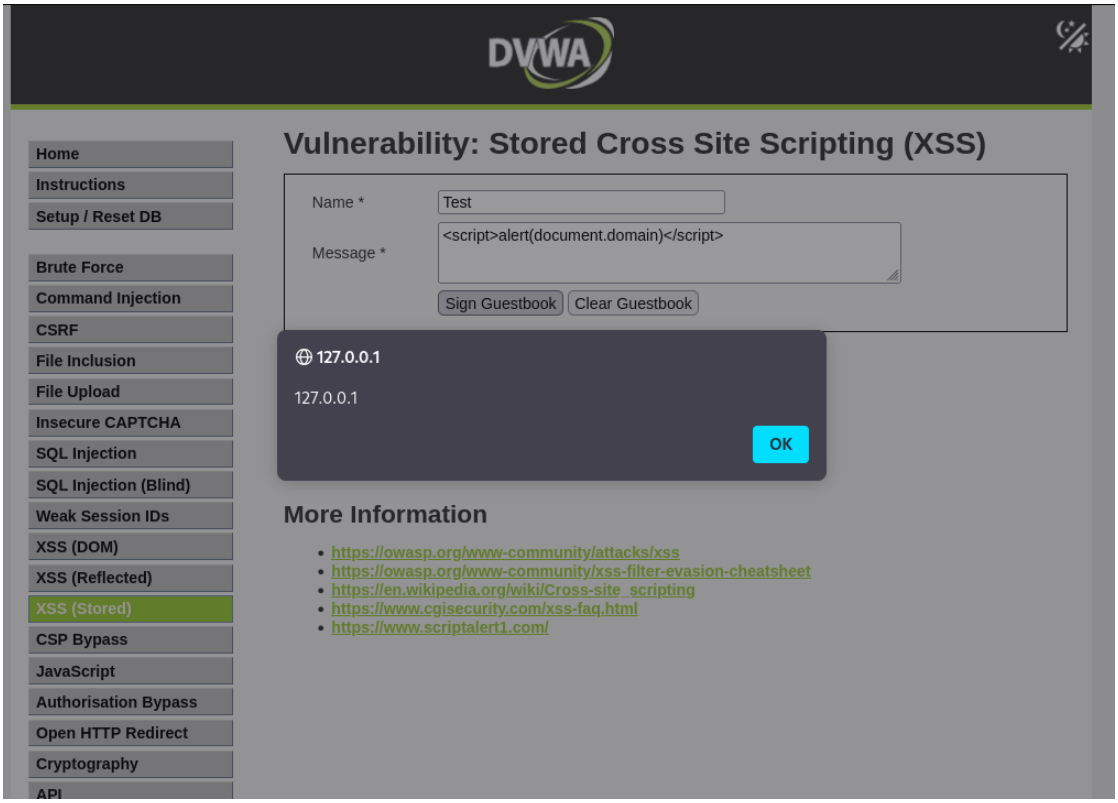


Figure 8. Stored XSS execution evidence.

Impact: Successful XSS can execute attacker-controlled script in a victim's browser origin, potentially affecting content, user actions or session-related data.

Mitigation: Context-aware output encoding, suitable input validation, restrictive CSP and appropriate cookie flags such as HttpOnly and Secure where applicable.

13. CSRF

The CSRF evidence shows a password-change request captured and inspected in Burp Suite. The controlled test demonstrates how a state-changing request can be analyzed when robust request validation is absent.

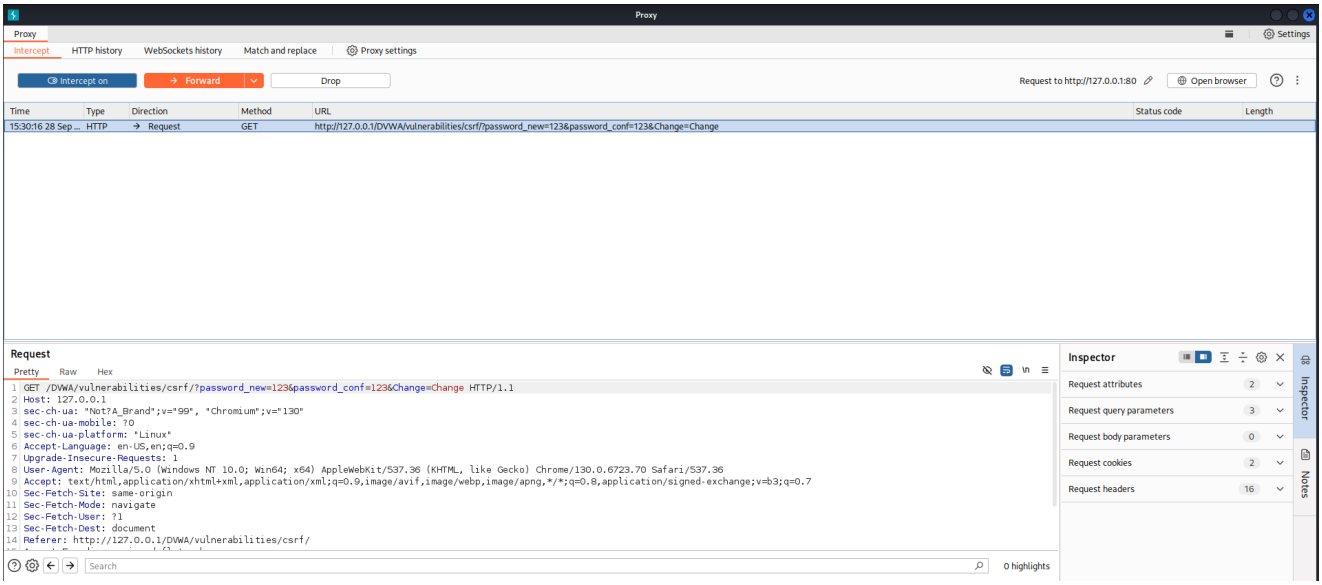


Figure 9. CSRF testing evidence captured through Burp Suite.

Impact: A vulnerable state-changing endpoint can cause an authenticated victim's browser to submit an unwanted action.

Mitigation: Use unpredictable session-bound anti-CSRF tokens and validate them on every state-changing request. SameSite cookies and appropriate Origin/Referer validation add defense.

14. LFI/RFI

The supplied evidence includes testing of DVWA file-inclusion functionality for local and remote inclusion scenarios. These tests demonstrate the risk of allowing user-controlled values to influence file-loading behavior.

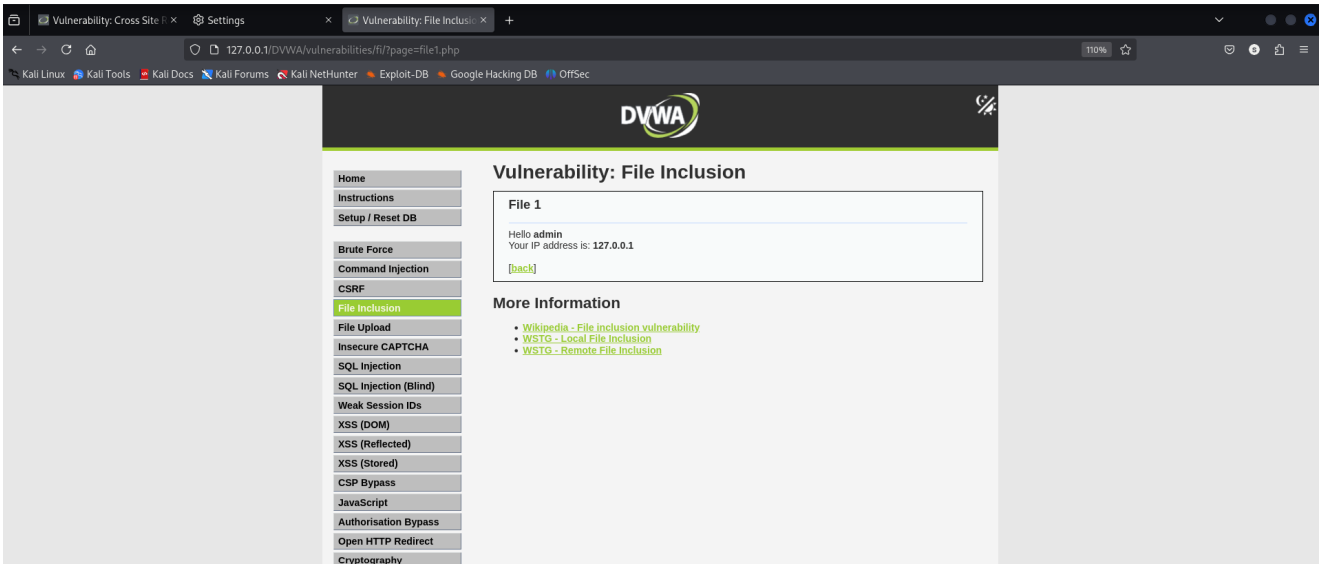


Figure 10. LFI module evidence.

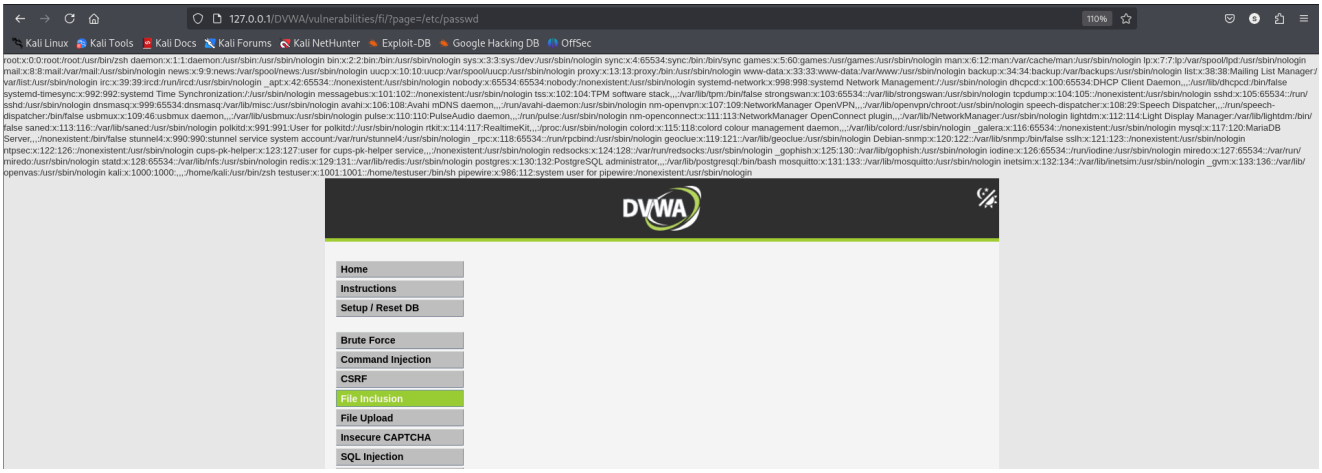


Figure 11. Additional file-inclusion evidence.

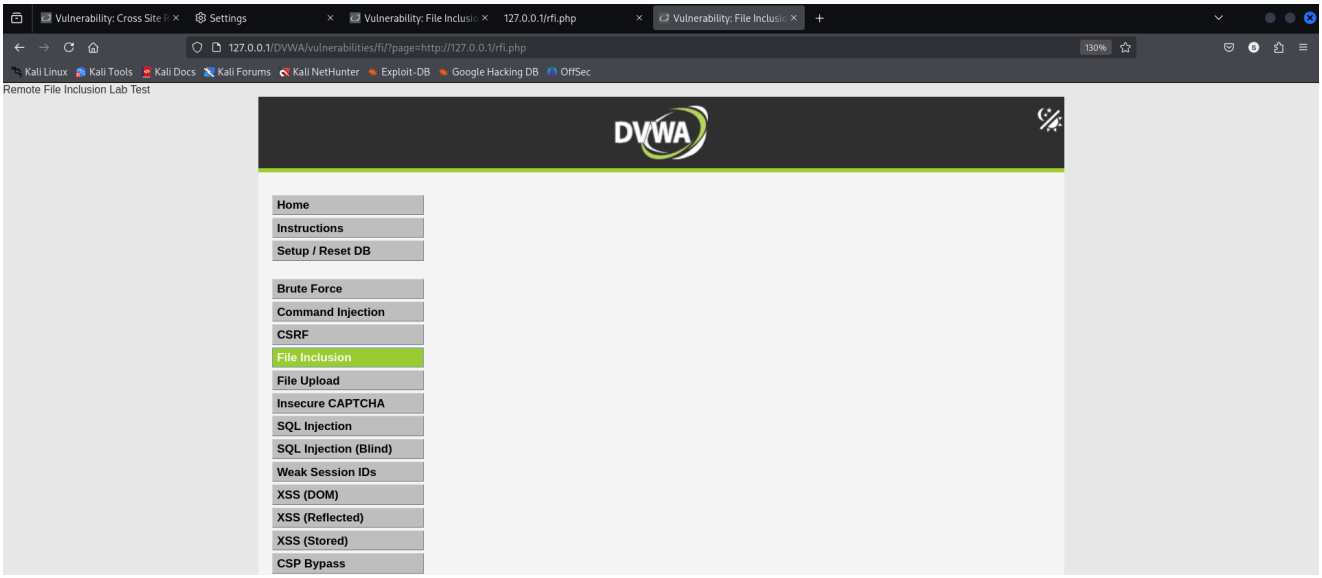


Figure 12. RFI module evidence.

Mitigation: Use strict server-side allowlists, canonicalize and validate paths, avoid mapping user input directly to filesystem locations, disable unnecessary remote inclusion and enforce least-privilege permissions.

15. Burp Suite & Security-Header Review

Burp Suite evidence includes login/request inspection, request modification, Intruder activity and security-header review. These captures support the controlled CSRF and HTTP-security analysis performed in the lab.

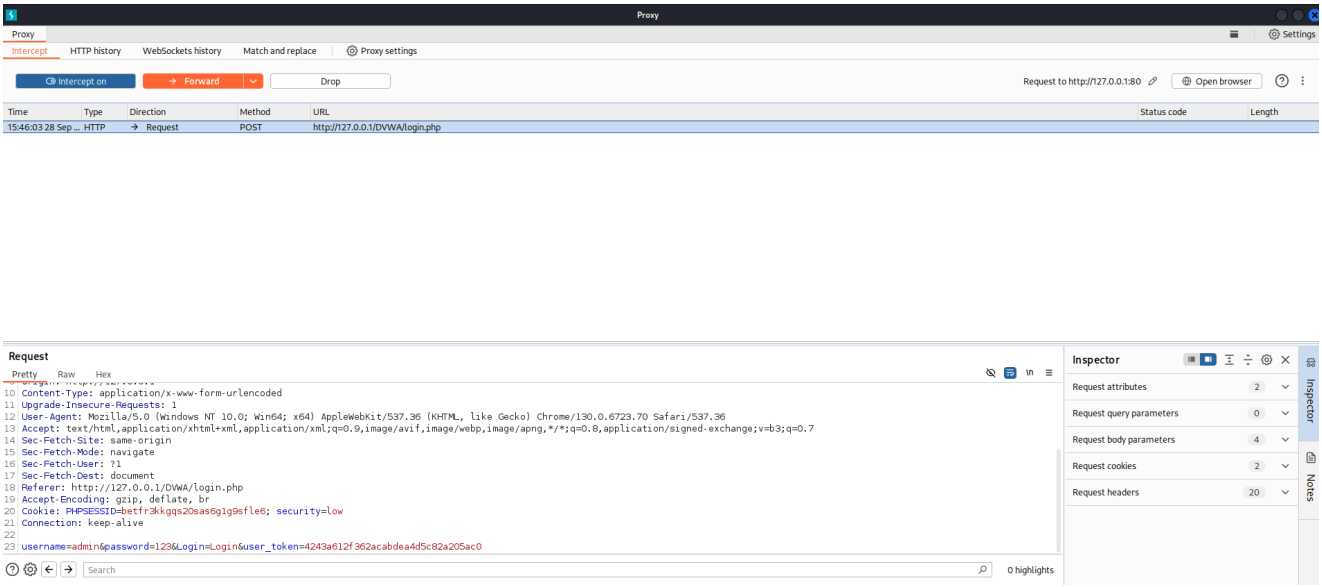


Figure 13. Burp Suite login/request inspection.

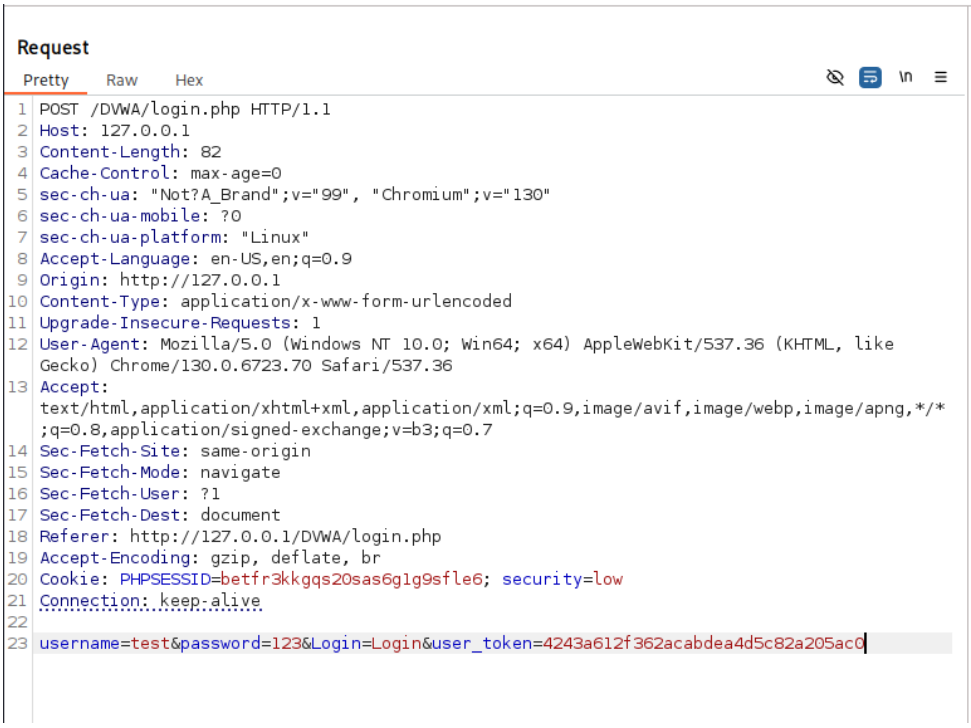


Figure 14. Controlled request modification.

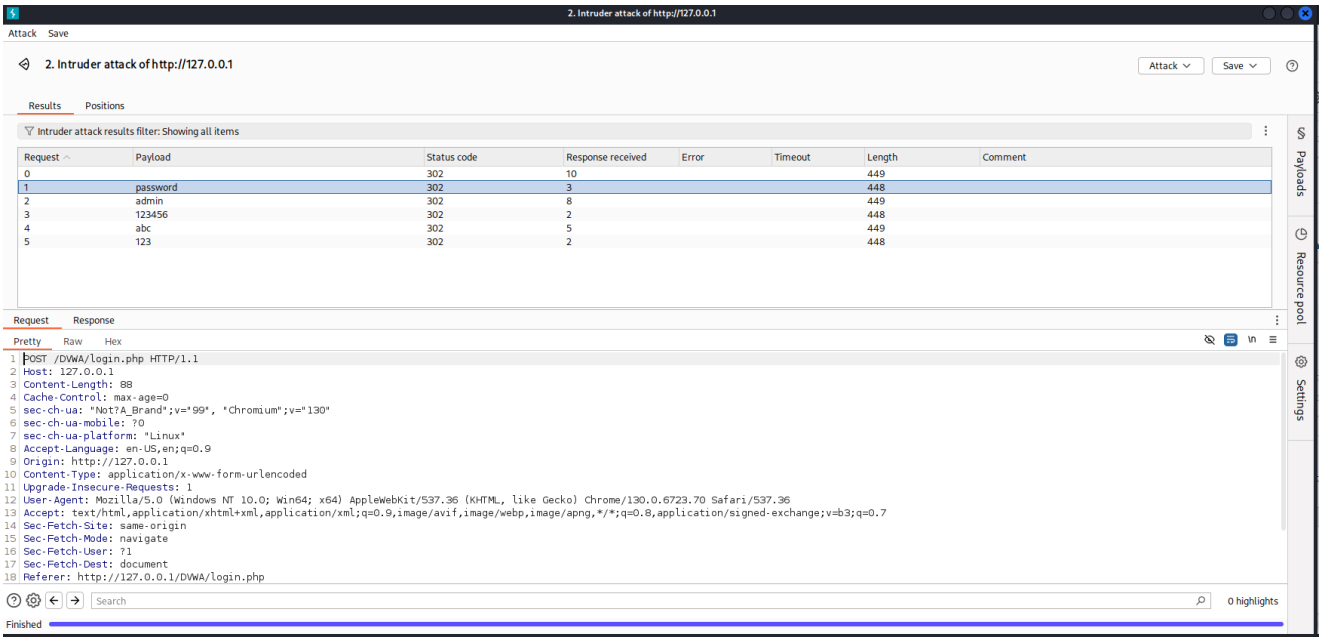


Figure 15. Burp Suite Intruder interface.

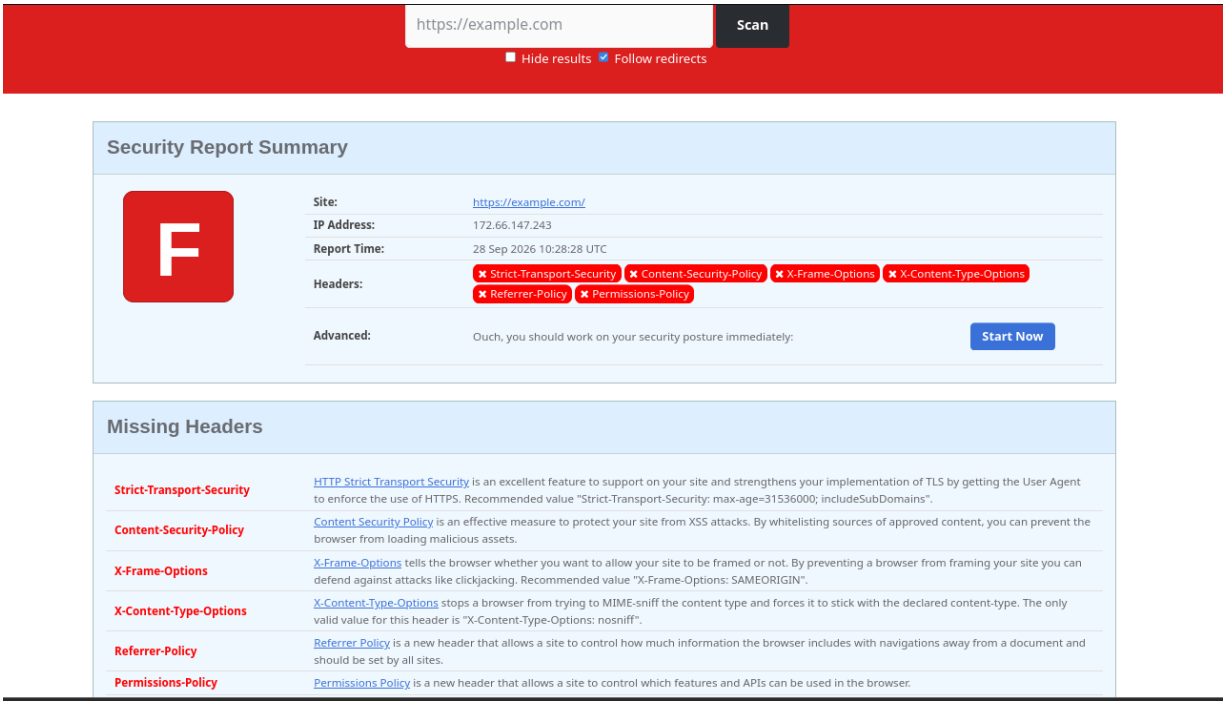


Figure 16. Security-header assessment output.

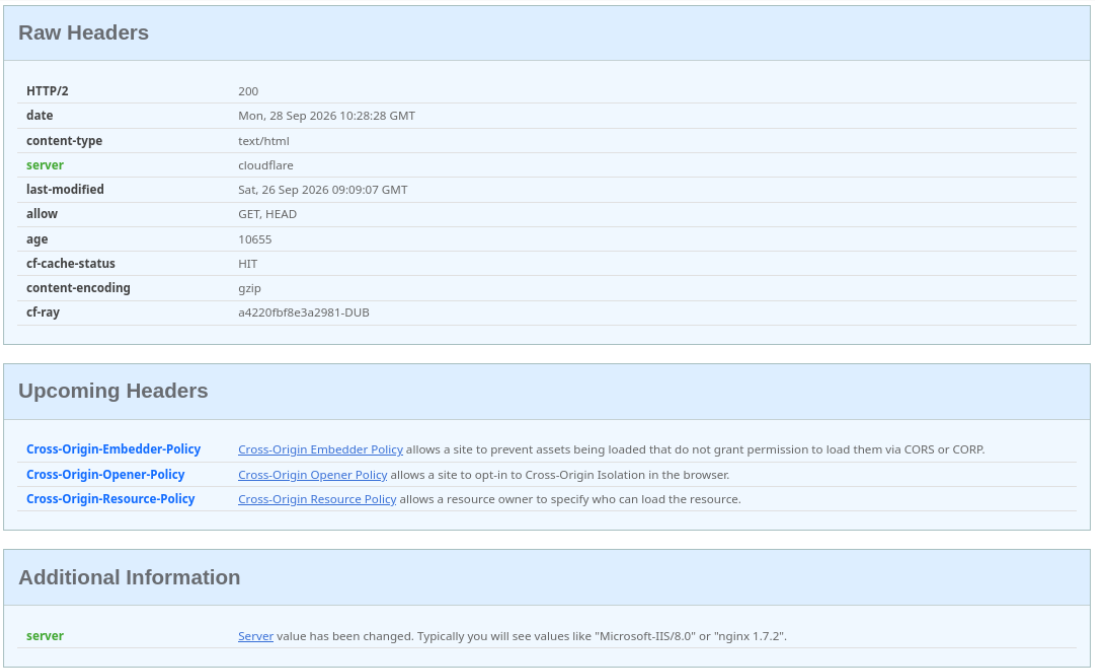


Figure 17. Response-header information.

Recommendation: Establish a security-header baseline appropriate to the application, including controls such as CSP, frame-ancestors/X-Frame-Options where appropriate, Referrer-Policy, Permissions-Policy, HSTS for HTTPS deployments and secure cookie attributes.

16. Incident Response

The previously demonstrated SQL Injection activity was used as a simulated incident. The goal was to show an end-to-end response workflow without introducing any external target.

16.1 Detection

Apache access logs captured the exact encoded request generated during the controlled SQL Injection test. The request originated from 127.0.0.1 and targeted the DVWA SQLi endpoint. The request parameter contained URL-encoded characters corresponding to a SQLi-style OR expression.

```
┌──(kali㉿kali)-[~]
└─$ sudo tail -n 30 /var/log/apache2/access.log
127.0.0.1 - - [05/Oct/2026:01:10:21 +0530] "GET /DVWA/vulnerabilities/sqli/ HTTP/1.1" 200 1938 "-" "Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0"
127.0.0.1 - - [05/Oct/2026:01:10:25 +0530] "GET /DVWA/security.php HTTP/1.1" 200 2327 "http://127.0.0.1/DVWA/vulnerabilities/sqli/" "Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0"
127.0.0.1 - - [05/Oct/2026:01:10:27 +0530] "GET /DVWA/vulnerabilities/sqli/ HTTP/1.1" 200 1937 "http://127.0.0.1/DVWA/security.php" "Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0"
127.0.0.1 - - [05/Oct/2026:01:10:29 +0530] "GET /DVWA/vulnerabilities/sqli/?id=1&Submit=Submit HTTP/1.1" 200 1960 "http://127.0.0.1/DVWA/vulnerabilities/sqli/" "Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0"
127.0.0.1 - - [05/Oct/2026:01:10:35 +0530] "GET /DVWA/vulnerabilities/sqli/?id=1%27+OR+%271%27%3D%271&Submit=Submit HTTP/1.1" 200 2019 "http://127.0.0.1/DVWA/vulnerabilities/sqli/?id=1&Submit=Submit" "Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0"

┌──(kali㉿kali)-[~]
└─$ sudo grep -iE "OR|UNION|SELECT|%27|%3D" /var/log/apache2/access.log | tail -n 20
127.0.0.1 - - [05/Oct/2026:01:10:35 +0530] "GET /DVWA/vulnerabilities/sqli/?id=1%27+OR+%271%27%3D%271&Submit=Submit HTTP/1.1" 200 2019 "http://127.0.0.1/DVWA/vulnerabilities/sqli/?id=1&Submit=Submit" "Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0"
```

Figure 18. Apache access-log activity around the simulated incident.

```
┌──(kali㉿kali)-[~]
└─$ cat ~/task5_sqli_incident.log
127.0.0.1 - - [05/Oct/2026:01:10:35 +0530] "GET /DVWA/vulnerabilities/sqli/?id=1%27+OR+%271%27%3D%271&Submit=Submit HTTP/1.1" 200 2019 "http://127.0.0.1/DVWA/vulnerabilities/sqli/?id=1&Submit=Submit" "Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0"
```

Figure 19. Filtered Apache log entry identifying the encoded SQLi request.

Event details: Source 127.0.0.1; 05/Oct/2026 01:10:35 +0530; GET; /DVWA/vulnerabilities/sqli/; HTTP 200. The event was preserved as **task5_sqli_incident.log**.

16.2 Containment

The vulnerable SQLi component was isolated by moving the application directory from **sqli** to **sqli_disabled**. This reduced exposure while remediation was prepared.

```
┌──(kali㉿kali)-[~]
└─$ sudo mv /var/www/html/DVWA/vulnerabilities/sqli /var/www/html/DVWA/vulnerabilities/sqli_disabled

┌──(kali㉿kali)-[~]
└─$ ls /var/www/html/DVWA/vulnerabilities/
api         brute    cryptography  csrf  fi        help.js     open_redirect  sqli_disabled  view_help.php        view_source.php  xss_d  xss_s
authbypass  captcha  csp           exec  help.css  javascript  sqli_blind     upload         view_source_all.php  weak_id          xss_r
```

Figure 20. SQLi component isolated as sqli_disabled.

16.3 Eradication / Remediation

The secure backup was restored to the SQLi implementation. The verified code contains **mysqli_prepare**, demonstrating prepared-statement use rather than direct concatenation of user input into SQL syntax.

```
┌──(kali㉿kali)-[~]
└─$ sudo grep -nE "mysqli_prepare|mysqli_stmt_bind_param" /var/www/html/DVWA/vulnerabilities/sqli/source/low.php
13:            $stmt = mysqli_prepare($GLOBALS["___mysqli_ston"], $query);
19:            mysqli_stmt_bind_param($stmt, "s", $id);
```

Figure 21. Prepared-statement implementation verified.

16.4 Recovery & Verification

The SQLi directory was restored to its original application path and Apache was restarted. The secure implementation was then verified again, closing the response cycle with a remediation check.

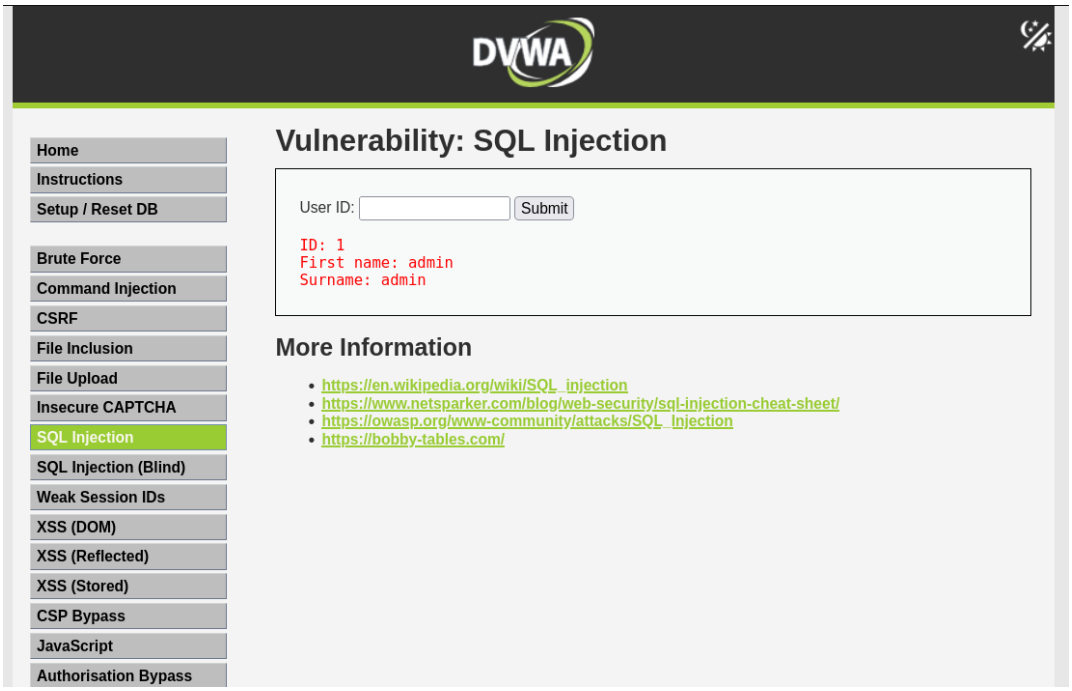


Figure 22. DVWA SQLi module in the controlled recovery workflow.

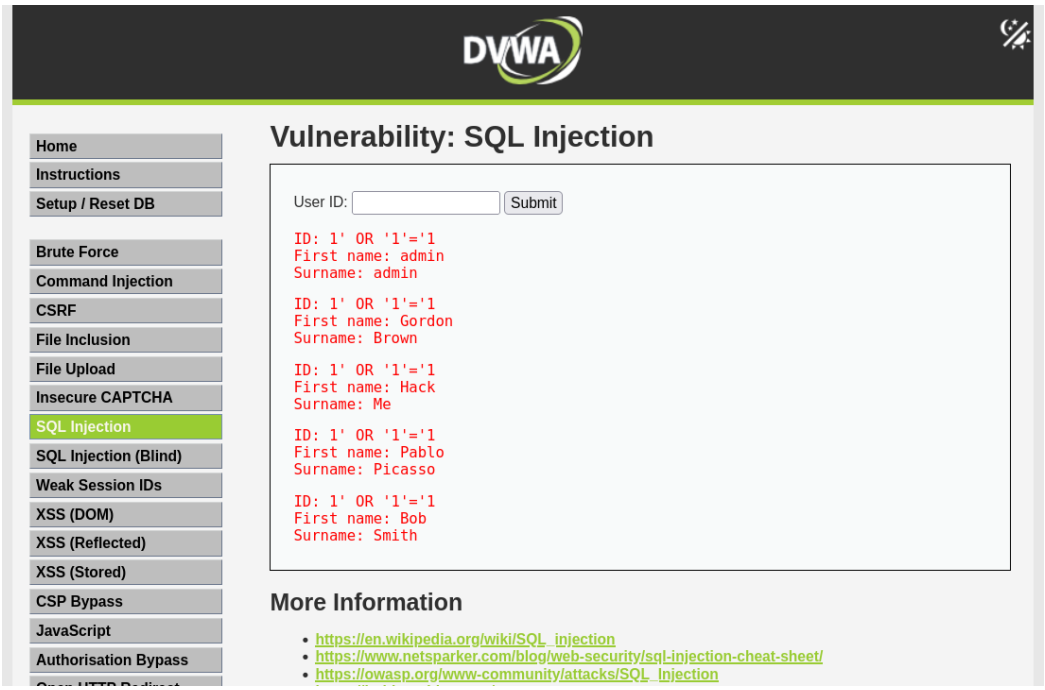


Figure 23. SQLi test state in the controlled workflow.

16.5 Post-Incident Timeline

Phase	Action	Evidence
Preparation	Vulnerable SQLi behavior demonstrated	DVWA SQLi evidence
Detection	Apache log reviewed for SQLi indicators	Figures 18–19; incident log
Containment	SQLi module moved to sqli_disabled	Figure 20
Eradication	Prepared-statement backup restored	Figure 21
Recovery	Application path restored; Apache restarted	Recovery workflow
Verification	Secure implementation confirmed	Prepared-statement verification

16.6 Lessons Learned

- Searchable web-server logs can provide useful detection evidence when suspicious parameters are retained.
- Prepared statements are a primary defensive control against SQL Injection.
- Containment should reduce exposure quickly while preserving evidence.
- Security controls should be re-tested after remediation.
- Centralized monitoring and alerting would improve detection speed in a production environment.

17. Findings & Risk Assessment

Risk ratings are qualitative prioritization values for this project. Formal CVSS scoring can be added if required by the academic rubric.

ID	Finding	Risk	Observed Evidence / Impact	Primary Mitigation
V-01	SQL Injection	High	Crafted input altered query behavior and exposed additional database information, validating	Prepared statements; input validation; least-privilege DB.
V-02	Reflected / Stored XSS	Medium	Script payloads executed in reflected and stored scenarios.	Context-aware output encoding; CSP.
V-03	CSRF	Medium	Password-change request captured/inspected in Burp, demonstrating state-changing risk.	Anti-CSRF tokens; SameSite; origin checks.
V-04	LFI/RFI	High	File-inclusion modules were exercised and evidence captured.	Strict allowlists; path canonicalization; disable unnecessa
V-05	Security-header weaknesses	Medium	Supplied header review showed missing/weak browser security controls.	Establish and continuously validate a security-header bas

18. Mitigation & Recommendations

- **SQL Injection:** Prepared statements, server-side validation, least-privilege DB accounts and safe error handling.
- **XSS:** Context-aware output encoding, suitable validation, restrictive CSP and secure cookie flags.
- **CSRF:** Session-bound anti-CSRF tokens on every state-changing request, SameSite cookies and appropriate origin validation.
- **LFI/RFI:** Allowlists, canonicalization, safe resource mapping and disabling unnecessary remote inclusion.
- **Security headers:** Establish a baseline and continuously validate headers during deployment.
- **Logging:** Centralize Apache logs, normalize URL-encoded indicators and alert on repeated injection patterns.
- **Secure SDLC:** Add SAST/DAST, dependency review, security-focused code review and regression testing.
- **Incident response:** Maintain playbooks for detection, containment, remediation, recovery and post-incident review.

19. Conclusion

The capstone demonstrates a complete controlled workflow for web application security testing and incident response using DVWA. It combines Nmap reconnaissance, existing vulnerability-testing evidence, Burp Suite analysis and a focused SQL Injection incident-response exercise.

The incident workflow connected offensive and defensive security operations: the SQLi request was detected in Apache logs, the affected module was contained, the secure prepared-statement implementation was restored, the application path was recovered and the remediation was verified. This provides a practical foundation for a SOC/DevSecOps workflow in which secure coding, logging, monitoring and incident playbooks operate together.

20. Evidence Appendix

The following representative screenshots were supplied with the project and are embedded in this report. The original evidence archives should be retained alongside the final GitHub submission.

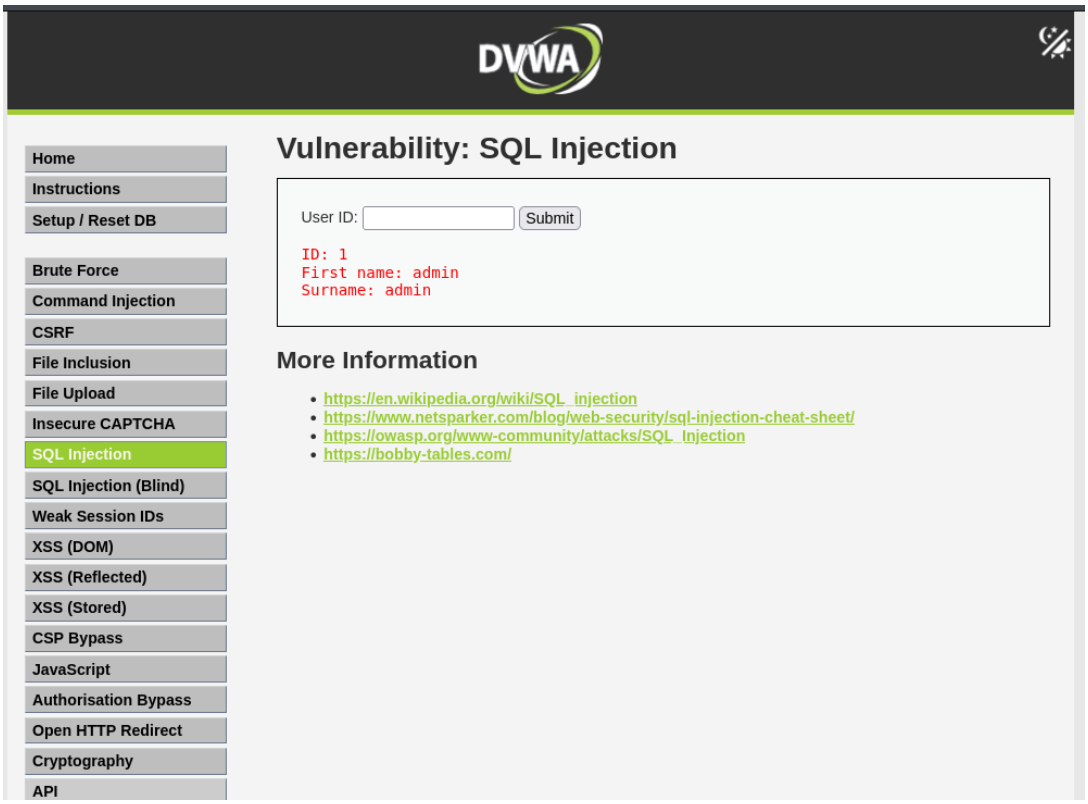


Figure 24. DVWA application interface in the local laboratory.

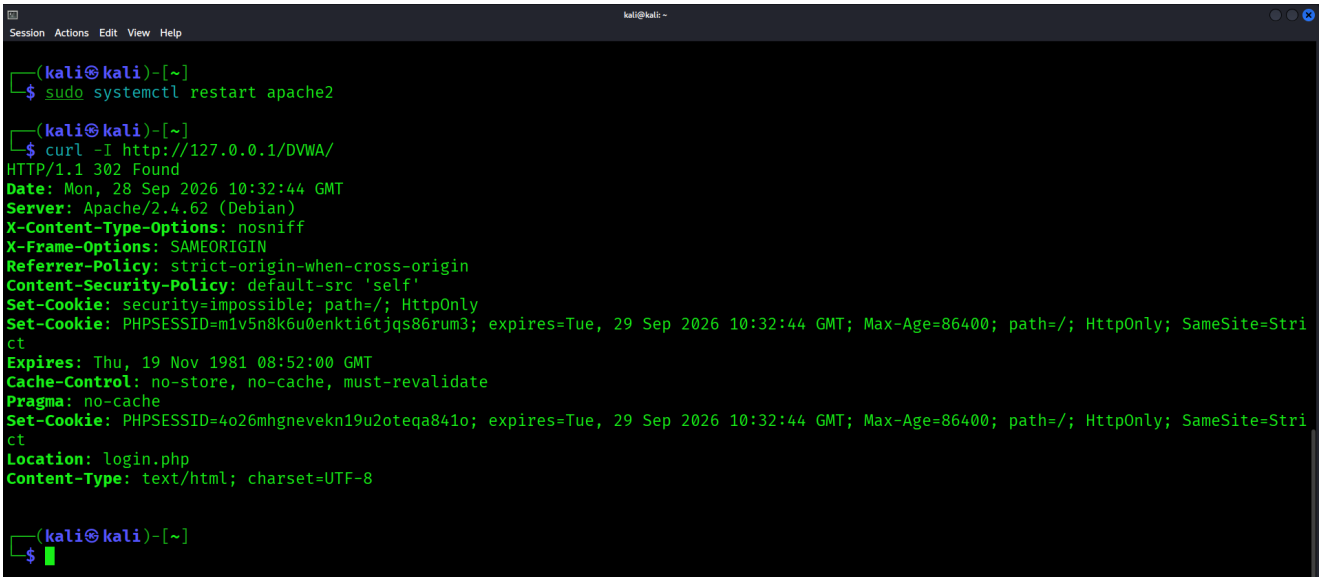


Figure 25. HTTP/security-header information observed on DVWA.

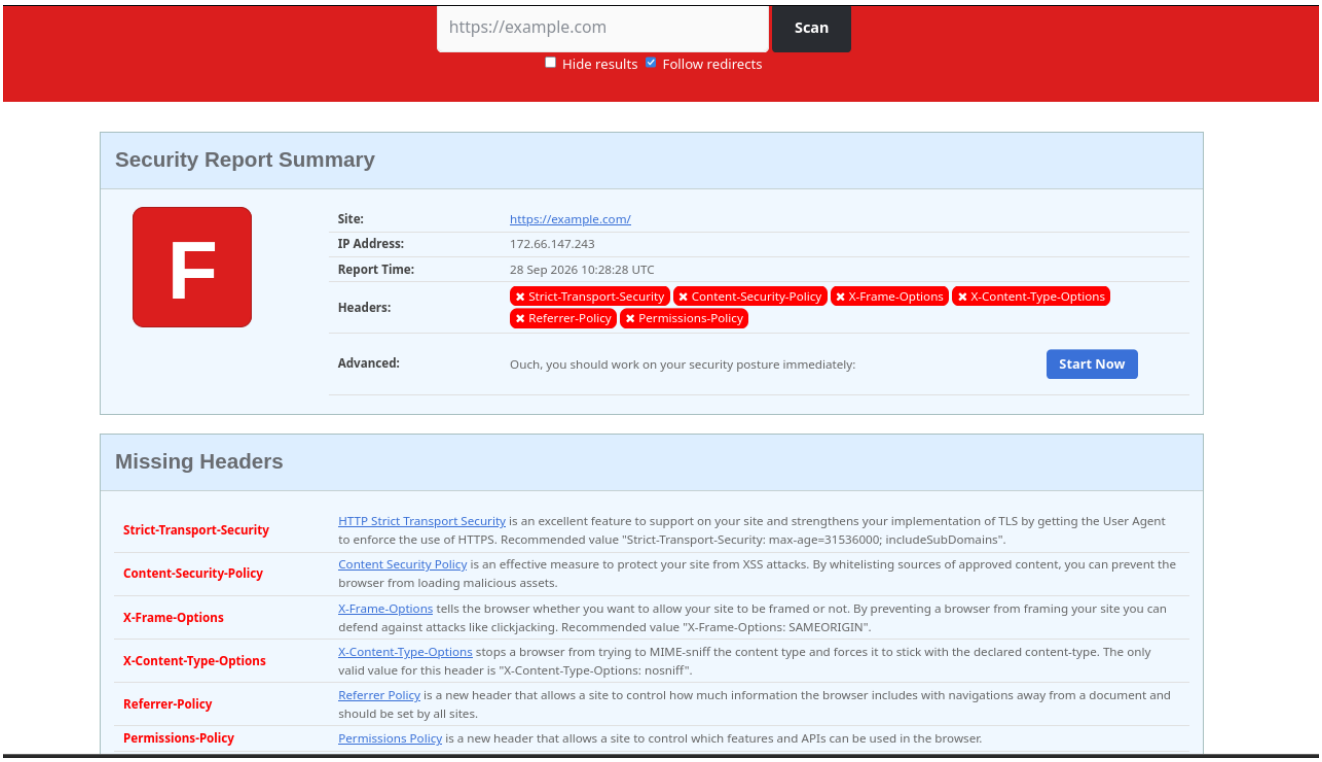


Figure 26. Security-header assessment output.

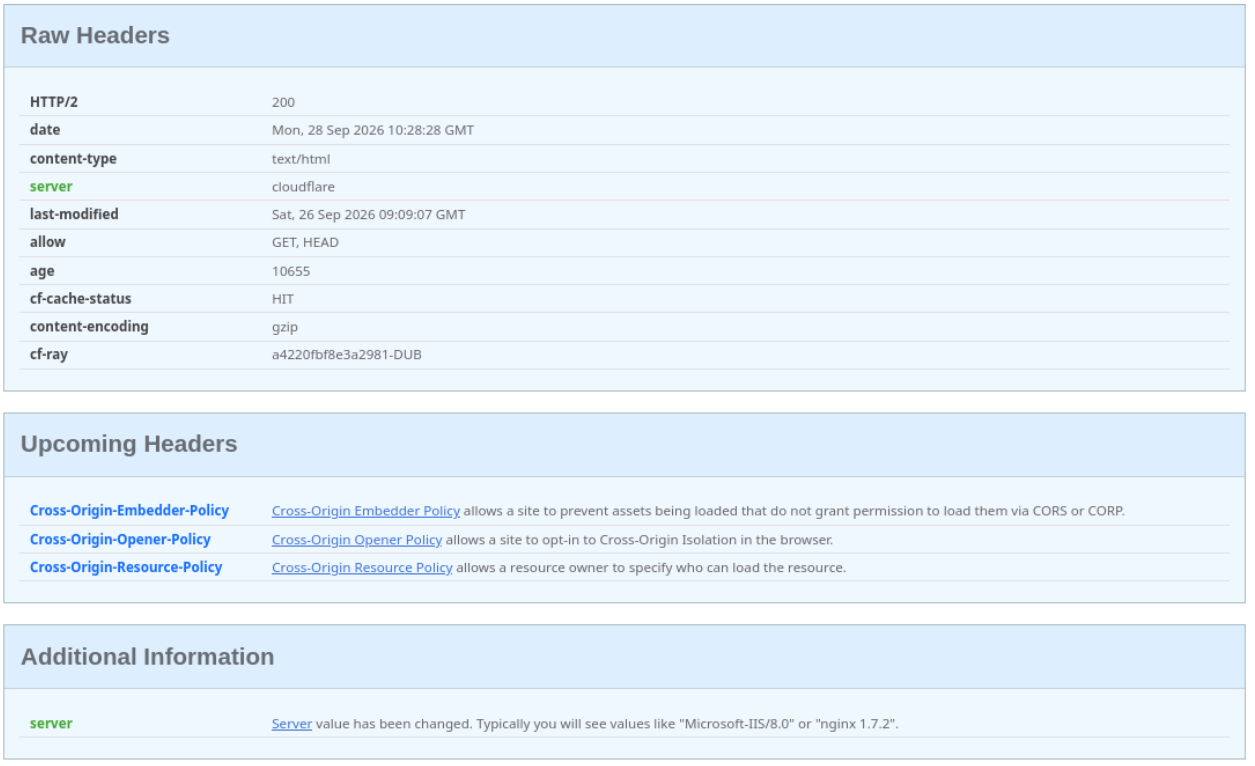


Figure 27. Detailed response-header information.

Evidence Index

#	Evidence artifact
1	DVWA.png
2	sql injection.png
3	sql pass hash.png
4	mitagation sql.png
5	refected xss.png
6	store xss.png
7	csrf.png
8	lfi 1.png
9	lfi 2.png
10	rfi.png
11	burp login.png
12	burp modify login.png
13	intruder.png
14	security header.png
15	security header 2.png
16	header on dvwa.png
17	nmap -sV.png
18	-O -sV.png
19	tail.png
20	sqli log.png
21	ls var.png
22	mysqli_prepare.png
23	DVWA sqli 1.png
24	sqli 2.png

Evidence handling: All screenshots and logs were supplied from the controlled lab. No external production target was used.